Genetic Programming Hyper-Heuristics for the Job Shop Scheduling Problem with Interval Durations: Interpretable Rules that Generalize Across Instance Sizes

Hernán Díaz

Department of Computing, University of Oviedo, Campus de Gijón, Gijón, 33204, Spain

Abstract

Dispatching rules remain the method of choice for scheduling under tight time budgets, but hand-crafted rules leave substantial performance unexploited, and the automated design of rules has scarcely been explored for scheduling problems with non-deterministic processing times. We present a genetic programming (GP) hyper-heuristic for the job shop scheduling problem with interval durations (IJSP), in which operation durations are known only to lie within an interval and schedules are evaluated by interval arithmetic. The GP evolves priority expressions over an *interval-aware* terminal set that exposes both the worst-case values and the widths of the uncertain quantities, so that reacting to uncertainty itself is expressible. On the 70 interval versions of Taillard's benchmark, over 30 independent evolutions the resulting rules average a relative error of 18.99% ($\pm$1.33) with a single deterministic constructive pass, and the best evolved rule attains 17.71%—improving on the strongest deterministic constructive baseline (Giffler–Thompson with MWKR tie-breaking, 29.5%) on every single instance—and generalizes zero-shot across all seven instance size classes

Email address: diazhernan@uniovi.es (Hernán Díaz)

despite being evolved on only four training instances of a single size class (20×15). On a second benchmark of classical instances it is applied without any re-evolution and, wrapped in a GRASP-style randomized scheme, matches the published average of a genetic algorithm that searches each instance separately. Evolved rules are compact, human-readable expressions, and an automatic configuration study with irace shows the approach is robust to its own hyperparameters. A controlled ablation over 30 evolutions per arm characterizes *when* interval awareness pays: the width terminals do not improve the expected makespan—the worst-case bounds carry that signal—but under an objective that rewards predictability they yield significantly narrower makespan intervals, a trade-off the user can dial. We position the method against fixed rules, active schedule generators, and published metaheuristics, characterizing the computational class in which interpretable evolved rules operate.

Keywords: interval job shop scheduling, genetic programming, hyper-heuristics, dispatching rules, scheduling under uncertainty, robustness

1. Introduction

Real manufacturing environments rarely know the exact duration of an operation in advance. The interval job shop scheduling problem (IJSP) models this uncertainty in a minimal, distribution-free way: each processing time is only assumed to lie within a known interval, and schedules are built and evaluated with interval arithmetic, so that the makespan of a solution is itself an interval and solutions are compared by an interval ranking. Population metaheuristics for the IJSP—artificial bee colony variants and tabu search among them—achieve strong results, at the cost of building and evaluating

large numbers of complete solutions for every instance [2, 3].

At the opposite end of the computational spectrum live *dispatching rules*: priority functions that build a schedule in a single constructive pass by repeatedly selecting the next operation to start. Their appeal is threefold—speed, interpretability, and trivial deployment—but hand-crafted rules (SPT, MOR, MWKR and relatives) leave a large quality gap with respect to search-based methods, as the experiments reported later in this paper confirm.

For *deterministic* scheduling, a rich literature has shown that this gap can be narrowed substantially by evolving rules automatically with genetic programming (GP) [18, 19]. For scheduling under interval uncertainty, however, the automated design of dispatching rules remains, to the best of our knowledge, unexplored. Moving automated rule design to interval durations raises questions that simply do not arise in the deterministic case: which attributes a rule should read, when the quantities describing a candidate operation—its duration, its earliest start, the work remaining in its job—are intervals rather than numbers; how candidate operations should be prioritized, and the resulting schedules compared, given that intervals admit no natural total order; and whether the *magnitude* of the uncertainty carries information that is useful for dispatching, or it is the position of the intervals that drives the decision. Answering these questions empirically requires exposing uncertainty to the rule as first-class attributes, which is the starting point of this work.

This paper makes the following contributions:

1. A GP hyper-heuristic for the IJSP whose terminal set is *interval-aware*: alongside worst-case processing time, earliest start and work remaining, rules can read the *widths* of these quantities, so that uncertainty-

sensitive prioritization is expressible at all (Section 4).

2. An empirical study on the 70 interval Taillard instances showing that evolved rules (i) beat every deterministic constructive baseline by a wide margin at equal cost, (ii) generalize zero-shot across instance sizes—evolved on 20×15, best class 50×15—and (iii) transfer to a second benchmark of classical instances, where a randomized multi-start of the same rule matches a published per-instance metaheuristic (Section 6).
3. An analysis of *why* the rules work: an anatomy of the evolved expressions (Section 7.1); an ablation that isolates the contribution of the interval-width terminals and characterises *when* interval awareness pays—not under a makespan objective, where worst-case bounds suffice, but under an objective that rewards predictability, where the widths yield significantly narrower makespan intervals (Section 7.2); and an executional-robustness study showing that the evolved rule yields schedules whose realised makespan deviates less from its prediction than the baselines', increasingly so as uncertainty grows (Section 7.3).
4. An automatic configuration study (irace, 200 experiments) of the evolutionary hyperparameters, whose winning configuration is used throughout (Section 5.3).

The remainder of the paper is organized as follows. Section 2 reviews related work. Section 3 formalizes the IJSP. Section 4 presents the GP hyper-heuristic, from both the hyper-heuristic and the evolutionary viewpoints. Section 5 details the benchmark, the training protocol and the irace configuration study. Section 6 reports the empirical results, and Section 7

analyses them: rule anatomy, the interval-awareness ablation, executional robustness and limitations. Section 8 concludes.

2. Related Work

2.1. Job shop scheduling under interval uncertainty

Interval durations offer a distribution-free uncertainty model requiring only bounds, in contrast to stochastic and fuzzy approaches [27]: they are the natural choice when historical data are insufficient to estimate distributions and a decision maker can only supply a minimum and a maximum duration. Scheduling with interval processing times has been studied across machine environments—single machine [12], flow shop, and flexible job shop [14], among others surveyed in [13]—and also analytically, through measures of the problem uncertainty induced by the intervals and of the stability of optimal sequences [11].

For the job shop with makespan minimisation, which concerns us here, the literature is dominated by population-based metaheuristics. Lei's population-based neighbourhood search [10], which ranks interval makespans by a possibility degree, was an early reference point; it was subsequently outperformed by a genetic algorithm with a systematic study of interval rankings [1] and by artificial bee colony variants [2, 3], which report the best published results for this problem. Related objectives have also been addressed under interval uncertainty, notably tardiness minimisation [15]. All of these are iterative search methods: they build and evaluate a population of complete solutions and refine it over many iterations for each instance solved. Fast constructive methods for the IJSP have received far less attention; dispatching rules with interval-aware

lexicographic comparisons are the de facto baseline.

2.2. Dispatching rules for job shop scheduling

Priority dispatching rules are among the oldest and most widely deployed scheduling heuristics: at every decision point, a priority function ranks the operations that are ready to start and dispatches the best one, building a schedule in a single constructive pass. Classical rules score each ready operation by a single attribute, and we adopt their standard acronyms throughout this paper. *Shortest* and *longest processing time* (SPT, LPT) look at the duration of the operation itself. *Earliest starting time* (EST) prefers the operation that can begin soonest given the current state of its job and its machine. *Most work remaining* (MWKR) and *most operations remaining* (MOR) look instead at the job the operation belongs to, and favour the one with the largest amount of pending work, measured respectively in time units and in number of operations. The *critical ratio* (CR) weighs the work still to be done against the time available until a due date. Surveys catalogue more than a hundred such variants [7, 8]. A step above single-attribute rules, the algorithm of Giffler and Thompson (G&T) [4] restricts each choice to a machine's conflict set, guaranteeing *active* schedules and markedly improving plain rules when a priority criterion is used to break ties inside that set; we write G&T-SPT and G&T-MWKR for the resulting combinations. Stronger constructive procedures exist for the *deterministic* job shop, notably the shifting bottleneck heuristic [5], which repeatedly solves one-machine subproblems with heads and tails by means of Carlier's algorithm [6]. These fall outside the scope of the present work: their machinery rests on exact one-machine optimisation and critical-path arguments whose extension to interval arithmetic, where no natural total order is available, is a research

contribution in itself and remains open. Decades of practice consolidated two lessons: rules are unbeatable in speed, transparency and ease of deployment, but hand-crafting them plateaued—no fixed combination of attributes performs well across shop configurations and objectives [8]. That plateau is precisely what motivated the *automated* design of rules.

2.3. Automated design of dispatching rules

Genetic programming [16, 17] evolves a population of expressions, usually represented as trees, by applying selection and variation operators directly on their structure. GP hyper-heuristics apply this machinery to *priority expressions* over scheduling attributes, and constitute the standard approach to automated rule design for production scheduling [18, 19]. Evolved rules have been shown to outperform hand-crafted ones across job shop variants, with active research on surrogate fitness, feature selection, and multi-objective formulations. Beyond single rules, a productive line combines several evolved rules into *ensembles* that jointly construct or vote on a solution [22], and recent work explores alternatives to evolutionary search for the same design task—systematic tree search over the space of expression trees [23]—reporting rules of comparable quality but smaller size and better readability. All of this work targets deterministic (or dynamic-stochastic) settings. Closest to our setting, Karunakaran et al. [20] evolve rules for dynamic job shops with uncertain processing times, exposing summary statistics of observed deviations (an exponential moving average) to the rule; uncertainty there is realized stochastically during simulation, and rules are evaluated on sampled scenarios. A recent survey of learned optimisation for the job shop [21], covering both GP and DRL constructive approaches, records no work on interval-valued processing times. To the best of our knowledge, evolving

rules whose *inputs* are interval attributes—and whose fitness is computed by interval arithmetic rather than by sampling realizations—has not been explored before.

Finally, deep reinforcement learning provides an alternative family of learned constructive methods for the JSP [25, 26]; GP rules and DRL policies are increasingly studied as competing learning paradigms for the same construction task, and controlled comparisons between them under common protocols have been identified as a gap in the recent literature [21]. Both families occupy the same computational class at deployment: a single constructive pass.

3. The Interval Job Shop Scheduling Problem

An IJSP instance consists of n jobs and m machines. Job j is a chain of m operations $o_{j1} \prec o_{j2} \prec \cdots \prec o_{jm}$, each requiring a distinct machine $\mu(o_{jk})$ exclusively and non-preemptively. The duration of operation o is an interval $p_o = [\underline{p}_o, \overline{p}_o]$ with $0 < \underline{p}_o \leq \overline{p}_o$: the only assumption is that the realized duration lies within these bounds.

Schedule construction uses interval arithmetic. For intervals $a = [\underline{a}, \overline{a}]$ and $b = [\underline{b}, \overline{b}]$, the two operations required by makespan minimisation are

$$a + b = [\underline{a} + \underline{b},\ \overline{a} + \overline{b}], \tag{1}$$

$$\max(a, b) = [\max(\underline{a}, \underline{b}),\ \max(\overline{a}, \overline{b})], \tag{2}$$

both component-wise [1]. A solution determines a task processing order π (a permutation with repetition of jobs, decoded left to right). Following the notation of [1, 3], let $PM_o(\pi)$ and PJ_o denote the predecessors of task o in its machine (according to π) and in its job. The *semi-active* schedule [9]

starts each task at

$$\mathbf{s_o}(\pi) = \max\big(\mathbf{c}_{PJ_o}(\pi),\, \mathbf{c}_{PM_{o(\pi)}}(\pi)\big), \qquad \mathbf{c_o}(\pi) = \mathbf{s_o}(\pi) + \mathbf{p_o}, \tag{3}$$

so starting and completion times of all tasks are themselves intervals. The problem can be stated as

$$\min_{R} \mathbf{C_{max}} \tag{4}$$

$$\text{s.t.} \quad \mathbf{C_{max}} = \max_{1\le j\le n}\{\mathbf{c}_{o(j,m_j)}\}, \tag{5}$$

$$\underline{s}_{o(j,l)} \ge \underline{c}_{o(j,l-1)}, \quad \overline{s}_{o(j,l)} \ge \overline{c}_{o(j,l-1)}, \qquad 1 \le l \le m_j,\ 1 \le j \le n, \tag{6}$$

$$\underline{s}_o \ge \underline{c}_{o'} \vee \underline{s}_{o'} \ge \underline{c}_o, \quad \overline{s}_o \ge \overline{c}_{o'} \vee \overline{s}_{o'} \ge \overline{c}_o, \quad \forall o \ne o' : \mu(o) = \mu(o'), \tag{7}$$

where Eq. (5) defines the interval makespan as the component-wise maximum of the job completion times, Eq. (6) enforces precedence within each job and Eq. (7) forbids overlaps on each machine, in both interval components [1, 3]. The minimum in (4) is taken with respect to an interval ranking R, since intervals admit no natural total order. We rank schedules lexicographically, comparing upper bounds first and using the lower bound only to break ties:

$$\mathbf{a} \preceq \mathbf{b} \iff \overline{a} < \overline{b} \ \vee\ \big(\overline{a} = \overline{b} \wedge \underline{a} \le \underline{b}\big), \tag{8}$$

one of the standard rankings of the IJSP literature and the one found in [1] to yield the most robust schedules. All methods in this paper—the GP rules and every baseline—share one implementation of this decoder and evaluator, so reported differences cannot stem from evaluator discrepancies.

Performance is reported as the relative error of the expected makespan with respect to a crisp reference bound LB of each instance,

$$\mathrm{RE} = \frac{E[\mathbf{C_{max}}] - LB}{LB} \times 100, \qquad E[\mathbf{C_{max}}] = \frac{\underline{C}_{\max} + \overline{C}_{\max}}{2}, \tag{9}$$

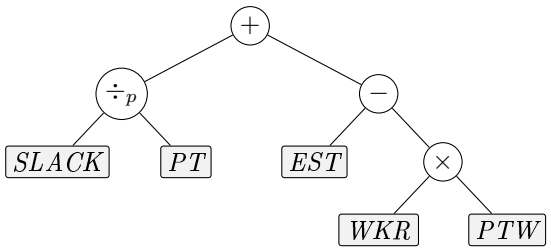

Figure 1: An individual is a priority expression tree; leaves are interval-aware attributes of the candidate operation (here the illustrative rule $SLACK/PT + EST - WKR \cdot PTW$). At each decision point the tree is evaluated on every eligible operation and the lowest-priority one is dispatched.

where $E[\mathbf{C_{max}}]$ is the expected value of the interval makespan under the uniform distribution on $[\underline{C}_{\max}, \overline{C}_{\max}]$, and LB is a bound of the underlying deterministic instance—Taillard's bounds for the Taillard-derived instances, the best known ones [2] for the classical set. It serves as an indicative common reference, not as a bound for the interval problem, for which none is available.

A schedule is eventually *executed* under one realization of the durations, and a second quantity of interest is how well its a-priori interval predicts that execution. Following [1], the $\bar{\varepsilon}$ robustness of a schedule with a fixed processing order is the average relative deviation of its executed makespan from the predicted expected value, over K realizations of the durations:

$$\bar{\varepsilon} = \frac{1}{K} \sum_{k=1}^{K} \frac{\left| C_{\max}^{ex,k} - E[\mathbf{C_{max}}] \right|}{E[\mathbf{C_{max}}]}, \tag{10}$$

where $C_{\max}^{ex,k}$ is the crisp makespan obtained by executing that same order when the durations turn out to take the values of realization k. A low $\bar{\varepsilon}$ therefore means that what was promised before execution is close to what is obtained after it.

Table 1: Interval-aware terminal set, with exact definitions. For an eligible operation o of job j on machine m: $p_o = [\underline{p}_o, \overline{p}_o]$ is its duration interval; C_j and C_m are the (interval) completion times of the last scheduled operation of job j and machine m; $\mathcal{R}_o$ is the set of operations of j after o; $\mathcal{E}$ is the current eligible set. Terminals are evaluated on *raw* (unnormalized) values.

Terminal	Definition	Role
PT	$\overline{p}_o$	worst-case processing time
PTW	$\overline{p}_o - \underline{p}_o$	duration uncertainty
EST	$\overline{e}_o = \max(\overline{C}_j, \overline{C}_m)$	worst-case earliest start
ESTW	$\overline{e}_o - \underline{e}_o$, with $\underline{e}_o = \max(\underline{C}_j, \underline{C}_m)$	start uncertainty
WKR	$\sum_{q \in \mathcal{R}_o} \overline{p}_q$	worst-case work remaining
WKRW	$\sum_{q \in \mathcal{R}_o} (\overline{p}_q - \underline{p}_q)$	remaining-work uncertainty
NOR	$\lvert\mathcal{R}_o\rvert$	operations remaining
SLACK	$\overline{e}_o - \min_{o' \in \mathcal{E}} \overline{e}_{o'}$	start slack within eligible set
ONE	1	constant

4. GP Hyper-Heuristic for the IJSP

We describe the method from two complementary viewpoints: as a *hyper-heuristic*—what an individual is and how it turns an IJSP instance into a schedule (Section 4.1)—and as an *evolutionary algorithm*—how the population of rules is initialized, varied and selected (Section 4.2).

4.1. Hyper-heuristic view: from expression trees to schedules

An individual is a priority expression tree (Figure 1). Leaves are drawn from the interval-aware terminal set of Table 1, and internal nodes from the function set

$$\mathcal{F} = \{ +,\ -,\ \times,\ \div_p,\ \min,\ \max,\ \mathrm{neg} \}, \tag{11}$$

where neg is the unary sign change and $\div_p$ denotes protected division, defined as $a \div_p b = a/b$ if $|b| > 10^{-9}$ and a otherwise. The rule is deployed inside a semi-active schedule generation scheme (SGS): starting from the empty schedule, at each of the N decision points the set $\mathcal{E}$ of eligible operations (those whose job predecessor is already scheduled) is formed, the tree is evaluated on the attributes of every $o \in \mathcal{E}$, and the operation with *lowest* priority value is appended at its earliest feasible start on its machine, Eq. (3).

Because the lowest value wins, a criterion that seeks to *maximise* an attribute must be expressed through the unary neg operator. Classic rules are thus points of this search space: shortest processing time is the single-node tree PT, whereas most work remaining is neg(WKR). We inject both as seed individuals in the initial population, so that the search departs from a population that already contains well-known rules rather than from purely random expressions.

Priority ties at dispatch time are broken by eligible set order (lowest job index first), which is deterministic; hence a rule maps each instance to exactly one schedule.

4.2. Evolutionary components

The initial population is built with the *ramped half-and-half* method, introduced by Koza [16] and still the standard choice for tree-based GP [17], which combines two tree-generation procedures over a range of depths. The *full* method expands every branch with function nodes until the prescribed maximum depth is reached, placing terminals only at that level, and thus produces complete trees whose leaves all lie at the same depth, whereas the *grow* method may close a branch earlier by placing a terminal —with probability 0.3 at each node in our implementation—and thus produces trees

of irregular shape and variable size. In both cases the function at an internal node is drawn uniformly from $\mathcal{F}$ and a leaf uniformly from the terminal set, the arity of the chosen function determining the number of branches to expand. Each individual draws its maximum depth uniformly from $\{2, 3, 4\}$ and is generated by either method with equal probability. The last two members of the population are not generated at random: they are the two seed rules of Section 4.1, inserted directly. Every individual is evaluated by the mean RE of its deterministic constructive pass over the four training instances, computed with the same environment, decoder and evaluator used by all baselines, which eliminates evaluator mismatch as a confound.

Each generation then proceeds as follows. Parents are chosen by tournament, drawing k individuals uniformly without replacement from the current population and returning the fittest of them. With probability p_c two parents are recombined by *subtree crossover* [16, 17], the standard recombination operator for tree-based GP. The offspring is a copy of the first parent in which one subtree has been replaced by a subtree taken from the second: a node is drawn uniformly at random in each parent, leaves included, and the subtree rooted at the node chosen in the first parent is substituted by the subtree rooted at the node chosen in the second. In its classical formulation the operator exchanges both subtrees and returns two offspring; we keep a single one per recombination. With the complementary probability $1 - p_c$ a single parent undergoes *subtree mutation* [16, 17]: a node is again drawn uniformly at random, and the subtree rooted at it is discarded and replaced by a freshly generated random tree, built with the grow method and maximum depth 3 over the full primitive set. Offspring whose size exceeds a cap of c nodes receive infinite fitness and are thereby excluded from selection. Note that the depth bounds above apply only to the trees created from scratch, at initial-

Table 2: Evolution parameters: the configuration selected by irace (Section 5.3), used for every experiment reported in this paper.

Population / generations	100 / 50
Initialization	ramped half-and-half (depths 2–4) + 2 seeds
Selection	tournament, size $k = 7$
Crossover / mutation prob.	$p_c = 0.77$ / 0.23 (subtree, depth ≤ 3)
Elitism	$e = 2$
Tree-size cap	$c = 30$ nodes (violations: fitness ∞)
Fitness	mean RE, deterministic rollout, TA11–TA14
Seeds	1–30 (RNG of evolution and environment)

ization and inside mutation: the depth of an individual is never constrained during the run, so recombination routinely produces expressions deeper than those of the initial population, the number of nodes being the only quantity that is bounded.

Replacement is fully generational, the offspring becoming the next population, except for the e best individuals of the current generation, which are copied into it unchanged. The process stops after a fixed number of generations and returns the best rule found. Algorithm 1 summarizes the generational loop and Table 2 lists the parameter values.

4.3. Randomized multi-start extension

A deterministic rule produces one schedule. For matched comparisons with sampling-based methods we wrap the evolved rule in GRASP-style randomization: with probability ε=0.1 the dispatched operation is drawn uniformly from the eligible set, otherwise the rule decides; sample 0 remains deterministic. Best-of-N then reports the lexicographically best schedule. This uniform perturbation of a single rule is the simplest member of a broader

Algorithm 1 GP hyper-heuristic for the IJSP

$P \leftarrow$ ramped half-and-half($pop - 2$) $\cup$ {SPT, MWKR}
evaluate(P) ▷ mean RE of one constructive pass on the training instances
for $g = 1$ **to** *gens* **do**
    $P' \leftarrow$ elite(P, e) ▷ elitism
    while $|P'| < pop$ **do**
        if rand() $< p_c$ **then**
            child $\leftarrow$ subtreeCrossover(tournament(P, k), tournament(P, k))
        else
            child $\leftarrow$ subtreeMutation(tournament(P, k))
        end if
        $P' \leftarrow P' \cup \{child\}$ ▷ fitness ∞ if size $> c$
    end while
    $P \leftarrow$ evaluate(P')
end for
return best rule in P

family of randomized rule-based builders; a richer alternative, orthogonal to our contribution, swaps among a *set* of evolved rules during construction [24].

5. Experimental Setup

5.1. Benchmark instances

We use two benchmarks, both taken from the IJSP literature [1, 2, 3]: the 70 interval versions of Taillard's instances (TA1–TA70, seven size classes from 15×15 to 50×20), and the 12 classical instances on which the published

IJSP metaheuristics report results (FT10, FT20, six instances of the La family and ABZ7–9, of sizes 10×10 to 20×15). Both were obtained from their crisp counterparts with the generation scheme introduced in [1]: for every operation a value δ is drawn uniformly at random in $[0, 0.15\,p]$, where p is its original duration, and that duration becomes the integer interval $[p - \delta, p + \delta]$, machine routes being unchanged. The interval is symmetric around p, so the crisp instance is exactly the midpoint scenario of its interval version. Since rules are evolved on Taillard instances only and applied to the classical set without any re-evolution, the latter doubles as a cross-family generalization test (Section 6.4). Both sets are released with this paper (see Data availability).

5.2. Training protocol, data split and baselines

Rules are evolved on TA11–TA14 (20×15), and TA15–TA20 serve as development set: they never contribute to fitness, but design and configuration decisions (including the irace study of Section 5.3) were selected on their performance, so we flag them as subject to development reuse. The remaining 60 Taillard instances and the 12 classical instances are fully unseen by every design decision and constitute the honest generalization test.

Each evolution uses population 100 and 50 generations: ∼20,400 fitness evaluations (≈30 min single-core) per seed; thirty independent seeds per configuration, following standard practice in evolutionary computation.

5.3. Hyperparameter configuration with irace

The four qualitative knobs of the evolution—tournament size, crossover probability, tree-size cap and elitism—were configured with irace over a budget of 200 experiments, each one a full evolution (population 100, 50 generations) on the training instances, scored by the resulting rule's RE on the

Table 3: Hyperparameter space explored by irace (200 experiments; cost = development-set RE of the evolved rule) and the winning configuration.

Parameter	Range / values	irace winner
Tournament size	$\{2, 3, 4, 5, 7\}$	7
Crossover prob. p_c	[0.5, 0.95]	0.77
Tree-size cap	$\{20, 30, 40, 60\}$	30
Elitism	$\{1, 2, 4\}$	2

development set. Table 3 lists the parameter space and the winning configuration, which is the one used for every experiment reported in this paper (Table 2). The elite configurations agree on a high selection pressure (tournament size 7) and a crossover probability in the range 0.72–0.77; the tree-size cap does not discriminate, with surviving elites spanning caps of 20, 30 and 40.

6. Results

6.1. Evolution behaviour and seed stability

Over 30 independent evolutions, training RE converges to 14.9 ± 1.5 (range 12.4–19.7), with most of the improvement realized by generation 25–30 (Figure 2). The evolved rules are structurally recognizable, combining the classical dispatching ingredients in interpretable ways. Figure 3 shows the best rule of the campaign, obtained with seed 1: 26 nodes over 12 leaves. Every terminal of the tree is non-negative by construction and processing times are at least one time unit, so both guards it contains resolve to the same branch at every decision, and the tree is exactly equivalent on the

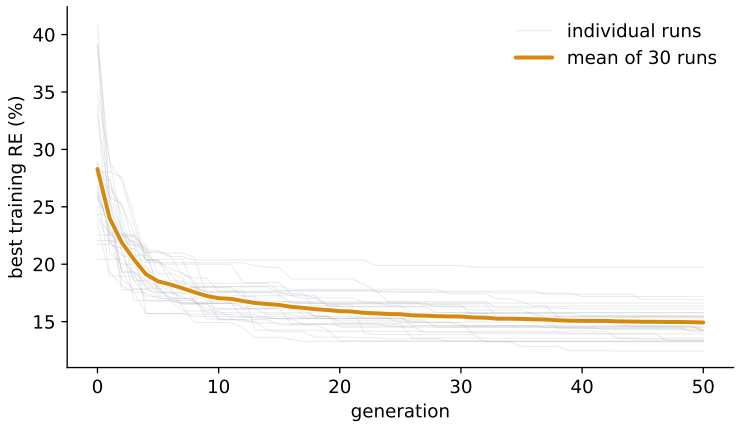

Figure 2: Best training fitness per generation over the 30 independent evolutions: each individual run and their mean.

problem domain to

$$SLACK^2 \; + \; 2\,PT \; - \; WKR \; - \; WKRW \; - \; 1, \tag{12}$$

an identity we verified over the 1.9 million dispatching decisions the rule takes on the benchmark. Since the lowest value is dispatched, the rule reads directly: $-WKR$ is most-work-remaining, $2\,PT$ is shortest-processing-time at double weight, $SLACK^2$ penalises quadratically any operation that cannot start at the earliest available time, and the constant is immaterial to the ranking. The remaining term, $-WKRW$, is the interval-aware one: among jobs with comparable remaining work the rule advances those whose remaining work is *more uncertain*, resolving that uncertainty early rather than carrying it to the end of the schedule. The evolved rule is thus a weighted blend of MWKR and SPT, regularised by a quadratic earliest-start term and tilted by the uncertainty of the work ahead.

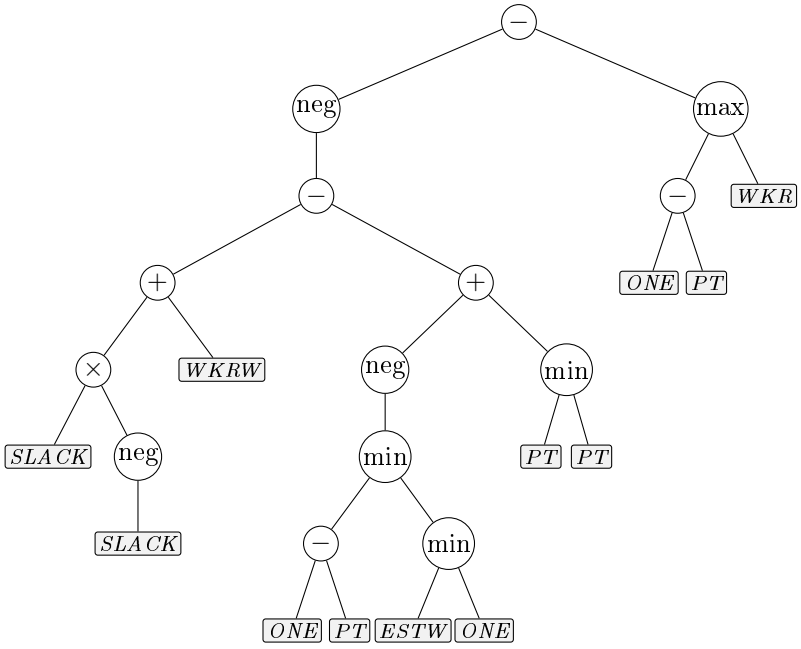

Figure 3: The best evolved rule of the campaign (seed 1), which attains 17.71% RE over the 70 instances. Circles are functions and shaded boxes the interval-aware terminals of Table 1; the operation with the lowest value of the expression is dispatched. The tree is shown exactly as evolved; Eq. (12) gives its simplified form.

6.2. Generalization to the full benchmark

Table 4 reports single-rollout RE per size class over all 70 instances, as mean ± standard deviation across the 30 evolved rules and for the single best rule. The best rule attains 17.71% global mean—evolved only on 20×15, its best class is the unseen 50×15 (13.8%), and the same pattern holds on average (13.7 ± 1.5). Because all terminals are per-operation attributes with no size-bound quantities, transfer requires no retraining or rescaling.

Table 4: RE (%) per size class, single deterministic rollout, over all 70 Taillard interval instances: mean ± sd across the 30 independently evolved rules, and the best evolved rule.

	15×15	20×15	20×20	30×15	30×20	50×15	50×20	All
Mean of 30	18.1±1.3	18.0±1.5	20.9±1.7	21.4±1.7	25.7±1.9	13.7±1.5	15.1±1.5	18.99±1.33
Best rule	15.7	16.6	18.1	21.1	24.1	**13.8**	14.6	**17.71**

6.3. Position among constructive methods

Table 5 reports every constructive baseline under the protocol of this paper—one deterministic pass per instance, the same decoder and evaluator for all methods—together with a uniformly random dispatcher as a reference point.

Two observations are needed to read the table correctly. First, the plain rules that ignore machine availability (SPT, LPT and the critical ratio CR, the latter being a due-date rule applied here without due dates) perform *worse than random* in this setting. This is a property of the schedule generation scheme, not a defect of the criteria: because a solution is decoded as a permutation whose operations are appended to their machines, systematically dispatching by processing time alone produces highly unbalanced permutations. The same criterion embedded in the Giffler–Thompson conflict set—which restores classical dispatching semantics—behaves as expected (SPT: 627% as a plain rule versus 70.6% under G&T). Second, the criteria that do account for the state of the shop (EST, MOR, MWKR) are comfortably better than random, and the strongest baseline overall is G&T-MWKR at 29.5%, which is the reference we use throughout.

Against that strongest baseline, the evolved rules average 18.99 ± 1.33 RE over the 30 evolutions, and the best rule attains 17.71%.

The gap is not marginal and not an artefact of averaging: the best evolved

Table 5: Constructive baselines on the 70 interval Taillard instances. All the rules are deterministic and build one schedule per instance, so the reported figures are the mean RE (%) over the 70 instances and the standard deviation *across instances*, which measures how uniform a method is over the benchmark rather than any run-to-run variability. The only stochastic entry is the uniformly random dispatcher, included as a reference and averaged over 10 independent dispatches per instance. Rules above the random line ignore machine availability and degrade below it under permutation-based decoding (see text).

Method	RE (%)	sd
LPT	703.6	168.0
SPT	627.2	152.2
CR	625.7	140.6
Random dispatcher	*127.2*	*13.9*
G&T-SPT	70.6	10.0
MWKR	64.7	10.4
MOR	45.5	10.1
EST	42.3	10.6
G&T-MWKR	**29.5**	6.4
GP rule (best of 30)	**17.71**	5.23

rule beats *every* rule in the table on *every* one of the 70 instances, and a paired Wilcoxon signed-rank test rejects equality against each of them at $p < 10^{-6}$ ($z = -7.3$). The narrowest margin over the whole comparison is 3.9 points, against G&T-MWKR on a single 20×15 instance; against the best plain rule (EST) the smallest margin is 6.3 points. Per-instance figures are given in Table A.9 (appendix). Crucially, this quality comes at no evaluation penalty over hand-crafted rules: evaluating the evolved expression tree over the eligible set costs the same, within measurement noise, as evaluating MOR or G&T-MWKR (the three differ by under 15% in per-schedule con-

struction time in our implementation), since all reduce to one vectorized pass over eligible operations per decision. Both cost and quality thus place the evolved rule squarely with the constructive methods, while closing much of the quality gap that separated them from search-based methods; that latter comparison is deferred to Section 6.4, where published results are available on a common benchmark.

6.4. Cross-family generalization: the 12 classical instances

The 12 classical instances serve a double purpose: they test the evolved rule on instance families and sizes it has never seen—the rule was evolved on four interval Taillard 20×15 instances and is applied here without any re-evolution—and they enable a direct comparison with the *published* results of the IJSP metaheuristics on their own benchmark and metric (Table 6). Those methods belong to different algorithmic families—evolutionary computation and swarm intelligence: a genetic algorithm evolving a population of permutations [1] and three artificial bee colony variants [2, 3], each of which runs a separate search on every instance. Alongside the deterministic pass, Table 6 reports the randomized extension of Section 4.3 at best-of-64 and best-of-1024: N independent ε-randomized passes of the same rule, of which the best schedule is kept, with no refinement or recombination among samples.

The two families also differ in how they spend computation: best-of-1024 evaluates 1024 mutually independent samples per instance, whereas the metaheuristics evolve a population of 250 individuals [1, 2] through successive generations, each of which depends on the previous one. In wall-clock terms the picture is mixed. A single constructive pass takes ≈0.1 s on instances of this size in our Python implementation, against the 0.5–2.2 s reported

Table 6: RE (%) on the 12 classical interval instances: our methods (deterministic single pass and GP-ε best-of-N; rule evolved on interval Taillard 20×15, applied zero-shot) versus the published average results (30 runs per instance) of the IJSP metaheuristics GA [1], ABC$_{E3}$, fEABC [2] and ESABC [3]. LB is the best-known lower bound of the expected makespan.

		Ours (zero-shot)			Published (avg of 30)			
Inst.	LB	GP	GP-ε_{64}	GP-ε_{1024}	GA	ABC$_{E3}$	fEABC	ESABC
FT10	930	8.1	7.0	6.3	5.2	4.1	3.5	3.0
FT20	1165	9.1	7.6	6.2	4.4	1.7	1.8	1.8
La21	1046	14.1	12.5	9.6	5.0	5.0	4.2	4.0
La24	935	11.1	8.7	7.4	6.3	5.1	4.9	5.0
La25	977	11.3	11.3	6.6	5.1	3.9	3.4	2.7
La27	1235	16.0	9.7	9.2	10.2	4.7	4.6	4.1
La29	1152	14.3	12.2	11.4	14.2	8.6	7.4	7.0
La38	1196	14.3	10.4	8.0	9.2	6.9	6.1	5.8
La40	1222	13.0	12.3	8.1	8.7	4.2	4.6	4.1
ABZ7	656	16.2	13.3	11.5	12.5	7.3	6.6	6.7
ABZ8	645	19.9	18.8	16.1	18.5	12.1	11.1	10.9
ABZ9	661	19.6	18.9	18.8	18.0	13.1	11.6	11.2
Mean		13.9	11.9	9.9	9.8	6.4	5.8	5.5

for the genetic algorithm and the 1.9–6.8 s reported for fEABC on the same instances [2], both implemented in C++. Best-of-1024, however, requires about two minutes of sequential Python time and is therefore *not* faster than the published metaheuristics.

In terms of quality, GP-ε at best-of-1024 reaches 9.9% RE, against the 9.8% published average of the genetic algorithm: each method is better on exactly half of the instances, and a Wilcoxon signed-rank test on the paired

per-instance values finds no significant difference ($z = 0.24$). A rule evolved once, on another benchmark, is thus statistically indistinguishable from a metaheuristic that evolves a population for every instance it solves. The purpose-built ABC variants remain clearly ahead (5.5–6.4%), as expected of iterative search: the point of this comparison is not to compete with them but to locate the constructive frontier—a single evolved rule, transferred across families without retraining, covers a substantial part of the gap that used to separate dispatching rules from metaheuristics.

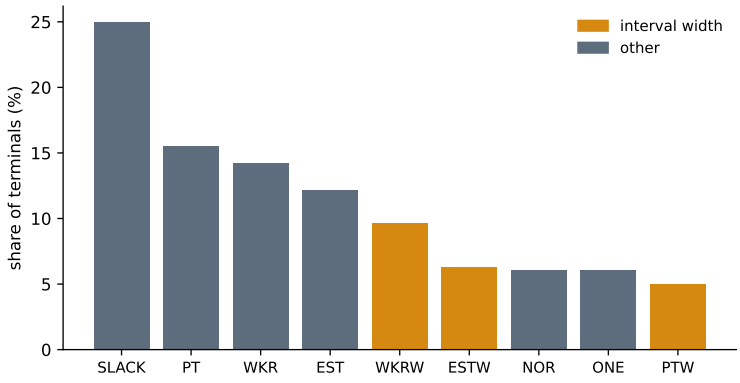

Figure 4: Terminal usage across the 30 evolved rules, as a share of all terminals. Interval-width terminals (amber) expose the magnitude of the uncertainty to the rule.

7. Analysis

Beyond aggregate quality, this section examines *why* the evolved rules work: what they are made of, whether the interval-specific ingredients actually contribute, how robust the resulting schedules are under executional uncertainty, and whether they survive re-evaluation under other uncertainty models.

7.1. Rule anatomy and interpretability

A practical appeal of GP over neural policies is that its output is a readable expression. Evolved rules are compact—mean tree size 33 nodes (19–40), mean depth 10—so a rule can be inspected and simplified by hand. Figure 4 reports how often each terminal is used across the 30 evolved rules. The backbone is the classical scheduling triad—slack ($SLACK$), work remaining (WKR) and earliest start (EST)—together with worst-case processing time (PT), confirming that the evolution rediscovers and refines known dispatching logic rather than exploiting benchmark artefacts. The *interval-width* terminals (PTW, $ESTW$, $WKRW$) account for ≈21% of all terminal usage and appear in 28 of the 30 evolved rules: the evolution consistently reaches for uncertainty information, even though worst-case bounds carry the larger share of the signal. Whether that width information translates into better schedules is quantified by the ablation of Section 7.2 and the robustness analysis of Section 7.3.

7.2. The value of interval-awareness: terminal ablation

The interval-width terminals are the distinctive ingredient of this setting, so we isolate their contribution by ablation: we re-run the full evolution—30 independent seeds per configuration—with and without the width terminals (PTW, $ESTW$, $WKRW$) in the terminal set, everything else held fixed, and compare the resulting rules with a paired Wilcoxon signed-rank test on the shared seeds. Table 7 collects the outcome.

[TODO: tab:ablation es lo unico que sigue en la configuracion por defecto: la fila 'full' del objetivo makespan da 18.68 ± 0.84, mientras que el resto del paper ya reporta el brazo tuneado (18.99 ± 1.33). Los cuatro brazos se

Table 7: Terminal ablation under two fitness objectives (30 independent evolutions per arm; rank-based tests between arms). *Width* is the relative width of the predicted makespan interval. Interval-width terminals do not help when the objective is the expected makespan—the two arms are then indistinguishable on both measures—but do produce significantly narrower schedules when the objective rewards predictability.

Fitness objective	Terminal set	RE (%)	Width (%)
Makespan $E[\mathbf{C_{max}}]$	full	18.68 ± 0.84	12.38 ± 0.31
	without widths	18.39 ± 0.59	12.38 ± 0.22
	test	*n.s.* ($z = 1.26$)	*n.s.* ($z = -0.12$)
Robust, Eq. (13)	full	19.64 ± 1.28	$\mathbf{12.01 \pm 0.74}$
	without widths	18.70 ± 0.89	12.42 ± 0.21
	test	—	$z = 2.73$, $p < 0.01$

rehacen con la campana tuneada en curso, que calcula RE y anchura de una sola pasada.]

Under the makespan objective, the width information turns out not to be used productively. Removing the three width terminals leaves performance unchanged: 18.39 ± 0.59 mean RE over the 70 instances without them versus 18.68 ± 0.84 with them, a difference that is not significant ($z = 1.26$) and, if anything, favours the smaller terminal set. The same holds for the resulting schedules: the relative width of their predicted makespan interval is identical to two decimals (12.38 ± 0.31 with the width terminals versus 12.38 ± 0.22 without them), and their $\bar{\varepsilon}$ robustness is indistinguishable (5.71 versus 5.73×10^{-3} for the best rule of each arm). So although the evolution does reach for the width terminals—they appear in 28 of the 30 rules (Section 7.1)—ablating them costs nothing: for minimising the expected makespan, the *position* of the intervals (their worst-case bounds) carries essentially all the

exploitable signal.

This negative result is, on reflection, unsurprising: the fitness being optimised rewards only the makespan, so there is no incentive for a rule to act on the magnitude of the uncertainty. We therefore repeat the ablation with a *robust* fitness that explicitly rewards predictability, minimising

$$f_\lambda = \overline{C}_{\max} + \lambda \left(\overline{C}_{\max} - \underline{C}_{\max} \right), \tag{13}$$

that is, a low worst case *and* a narrow interval, with the weight λ setting the balance between the two; we take $\lambda = 1$ here. Under this objective the ablation reverses, and the width information does pay. Rules evolved with the width terminals produce schedules whose makespan interval is significantly narrower than those evolved without them (12.01 ± 0.74 versus 12.42 ± 0.21 relative width; paired Wilcoxon $z = 2.73$, $p < 0.01$). The asymmetry is informative: deprived of width terminals, the evolution cannot narrow the interval below $\approx$12.4% *whatever* objective it is given, whereas with them it reaches 12.0% as soon as narrowness is rewarded. The gain is paid for in expected makespan (19.64 versus 18.70 RE), which is precisely the trade-off the robust objective asks for.

The weight λ of Eq. (13) governs that trade-off, and sweeping it traces the frontier between quality and predictability (Figure 5). Both quantities move monotonically with λ: as the weight grows from 0.5 to 4, the relative width of the makespan interval falls from 12.16 ± 0.51 to 10.30 ± 1.43 while the expected makespan degrades from 19.09 ± 1.45 to 23.04 ± 3.26 RE. At the far end of the sweep the schedules are therefore narrower than anything the ablated rules can produce, at a cost of some four points of RE. The practitioner can choose an operating point along this curve; deprived of the width terminals, the evolution is pinned at $\approx$12.4% *whatever* value of λ it is

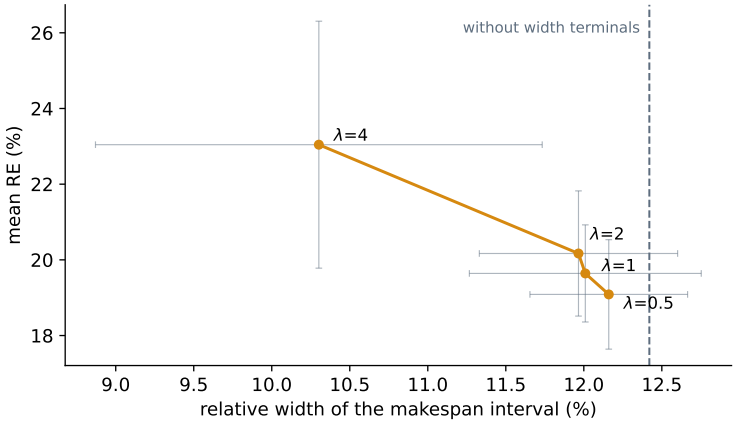

Figure 5: Quality–predictability frontier traced by the weight λ of the robust fitness, Eq. (13). Rules evolved *with* the interval-width terminals; each point is the mean over the independent evolutions of that arm, every rule evaluated on the 70 instances, and the grey bars span ± 1 sd in both coordinates. The dashed line marks the arm evolved *without* them, which cannot narrow the interval whatever λ is used. Increasing λ buys narrower makespan intervals at the price of a larger expected makespan.

given, so it has no such frontier available to it at all.

Together, these experiments characterise *when* interval awareness matters, rather than asserting that it always does: the width of an interval is actionable information for a dispatching rule, but only when the objective values predictability. A practitioner minimising expected makespan can safely use the worst-case attributes alone; one who needs schedules whose realised makespan is tightly predictable gains from exposing the widths, and can dial the balance through λ.

7.3. Robustness under executional uncertainty

Relative error measures a-priori quality; the $\bar{\varepsilon}$ robustness of Eq. (10) measures how well that a-priori prediction survives execution, and is the quantity

Table 8: Robustness over the 70 instances (lower is better on both measures). *Rel. width*: relative width of the predicted makespan interval, $(\overline{C}_{\max} - \underline{C}_{\max})/E[C_{\max}]$ (%). $\bar{\varepsilon}$ ($\times 10^3$): mean deviation of the executed makespan from its prediction, at nominal interval widths and widened by +20% and +40% (Monte Carlo, K=1000 realizations, common random numbers across methods).

		$\bar{\varepsilon}$ ($\times 10^3$)		
Method	Rel. width (%)	+0%	+20%	+40%
GP rule	**12.6**	**5.8**	**6.8**	**7.7**
G&T-MWKR	13.8	6.3	7.5	8.6
MOR	14.4	6.5	7.7	8.9

examined here. We estimate it by Monte Carlo, drawing K=1000 realizations of the durations uniformly on each operation's interval and executing the fixed processing order on each of them; lower is more robust. To probe behaviour under growing uncertainty we also widen every interval symmetrically about its midpoint by +20% and +40% (clipping negative bounds to zero), re-evaluating the same processing orders.

The evolved rule does not merely produce lower-makespan schedules—it produces *more robust* ones, on two complementary measures (Table 8). First, its predicted makespan interval is the *tightest*: a relative width of 12.6% against 13.8% (G&T-MWKR) and 14.4% (MOR), so the a-priori uncertainty attached to the schedule is smaller. Second, and more tellingly, its executed makespan tracks the prediction most closely: $\bar{\varepsilon}$ is lower than for both baselines at the nominal width (5.8 versus 6.5 for MOR and 6.3 for G&T-MWKR), and its advantage *widens* as uncertainty grows (at +40%: 7.7 versus 8.9 and 8.6). Figure 6 shows the per-instance $\bar{\varepsilon}$ distributions. It is worth being precise about the source of this advantage. The figure also car-

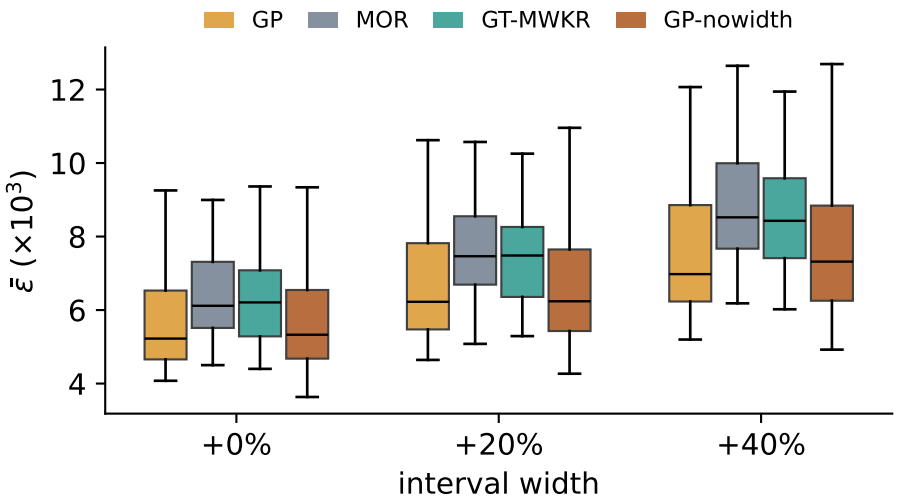

Figure 6: Per-instance $\bar{\varepsilon}$ robustness by method at each uncertainty level (lower is better). The evolved rule's robustness advantage over the constructive baselines grows with interval width. *GP-nowidth* is the ablated arm of Section 7.2, evolved without the three interval-width terminals: it is indistinguishable from the full rule, which locates the robustness advantage in the quality of the schedules rather than in the rule reading widths.

ries the ablated arm of Section 7.2: rules evolved *without* the interval-width terminals are as robust as the full rule at every uncertainty level ($\bar{\varepsilon}$ of 5.7, 6.8 and 7.8 against 5.7, 6.8 and 7.7), so the advantage does not come from the rule reading interval widths. It comes from producing better schedules in the first place: compact schedules with fewer and shorter critical paths accumulate less uncertainty along them, so both the predicted interval and the realised deviation shrink together.

[TODO: tras la campana: (1) tab:robustness sale de robustness_eps.csv y fig:robustness de robustness_ablation.csv, que son reglas GP distintas (diferentes en las 70 instancias, medias casi iguales): regenerar AMBAS de una

sola ejecucion con la mejor regla tuneada; (2) anadir la fila GP-nowidth a tab:robustness, que la figura ya muestra; (3) Wilcoxon pareado sobre las 70 instancias para GP vs baselines y GP vs GP-nowidth.]

7.4. Limitations

Three limitations bound the scope of our conclusions. First, all rules operate inside a semi-active SGS; active (insertion-based) generation schemes are known to help weaker constructors, and although our evolved rules produce near-active schedules already, co-evolving rules with an active SGS in the loop remains untested. Second, the uncertainty model is the $\pm 15\%$ symmetric interval scheme standard in the IJSP literature; other widths and asymmetric intervals may change the relative value of uncertainty information. Third, our runtime figures are relative (evolved rules cost the same as hand-crafted ones per pass); absolute times depend on the research-grade Python implementation and would shrink by orders of magnitude in an optimized production implementation, without affecting any ranking reported here.

8. Conclusions and Future Work

We presented what is, to the best of our knowledge, the first GP hyper-heuristic for job shop scheduling with interval durations, built on a terminal set that exposes interval widths as first-class attributes. The empirical conclusions are three. (i) Evolved rules dominate every deterministic constructive baseline on the 70-instance benchmark by a wide, statistically significant margin (18.99 ± 1.33 over 30 evolutions, 17.71% for the best rule, versus 29.5% for the strongest baseline—which the best rule beats on all 70 instances) at the cost of a single constructive pass, and (ii) they generalize zero-shot across

all size classes—evolved on 20×15, best on the unseen 50×15—because the representation contains no size-bound quantities. (iii) The evolutionary hyperparameters were configured automatically with a 200-experiment irace study, whose winning configuration is used throughout. Beyond aggregate quality, the analysis pins down *when* the interval-specific ingredients matter: exposing the widths of the uncertain quantities does not improve the expected makespan—the worst-case bounds already carry that signal—but it does let the evolution produce significantly narrower makespan intervals once the objective rewards predictability. Interval awareness is thus not a free improvement but a lever, available when the application values predictable schedules over nominally shorter ones.

Future work follows three lines. First, richer symbolic models: ensembles of complementary rules [22] and co-evolution of construction and tie-breaking. Second, using evolved rules as cheap, high-quality population initializers for the metaheuristics that lead the IJSP: the pools of diverse, high-quality sequences produced by GP-ε are directly usable as seeded initial populations, and quantifying the effect of such seeding is ongoing work. Third, a controlled comparison with learned neural constructive policies under matched training and inference budgets—the head-to-head evaluation that the recent literature [21] identifies as missing.

CRediT authorship contribution statement

Hernán Díaz: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Resources, Data curation, Writing – original draft, Writing – review & editing, Visualization, Project administration.

Acknowledgements

This work was supported by the Spanish Ministry of Science, Innovation and Universities (MCIN/AEI/10.13039/501100011033) under grant PID2022-141746OB-I00.

Declaration of competing interest

The author declares that there are no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The GP hyper-heuristic is implemented from scratch in pure Python/NumPy, with no evolutionary-computation dependencies; evolved rules are stored as plain JSON expression trees and evaluated as drop-in dispatching strategies by the same environment used by every baseline. The benchmark instances, the generated solutions and rules, and the analysis scripts are shared in a Zenodo repository. [TODO: DOI de Zenodo al depositar]

Appendix A. Per-instance results

Table A.9 lists, for each of the 70 interval Taillard instances, the crisp lower bound and the single-pass RE (%) of the three deterministic constructive methods.

References

[1] H. Díaz, I. González-Rodríguez, J. J. Palacios, I. Díaz, C. R. Vela. A genetic approach to the job shop scheduling problem with interval uncertainty. In *Information Processing and Management of Uncertainty in Knowledge-Based Systems (IPMU)*, pages 663–676. Springer, 2020.

[2] H. Díaz, J. J. Palacios, I. González-Rodríguez, C. R. Vela. Fast elitist ABC for makespan optimisation in interval JSP. *Natural Computing*, 22(4):645–657, 2023.

[3] H. Díaz, J. J. Palacios, I. González-Rodríguez, C. R. Vela. An elitist seasonal artificial bee colony algorithm for the interval job shop. *Integrated Computer-Aided Engineering*, 30(3):223–242, 2023.

[4] B. Giffler, G. L. Thompson. Algorithms for solving production-scheduling problems. *Operations Research*, 8(4):487–503, 1960.

[5] J. Adams, E. Balas, D. Zawack. The shifting bottleneck procedure for job shop scheduling. *Management Science*, 34(3):391–401, 1988.

[6] J. Carlier. The one-machine sequencing problem. *European Journal of Operational Research*, 11(1):42–47, 1982.

[7] S. S. Panwalkar, W. Iskander. A survey of scheduling rules. *Operations Research*, 25(1):45–61, 1977.

[8] R. Haupt. A survey of priority rule-based scheduling. *OR Spektrum*, 11(1):3–16, 1989.

[9] A. Sprecher, R. Kolisch, A. Drexl. Semi-active, active, and non-delay

schedules for the resource-constrained project scheduling problem. *European Journal of Operational Research*, 80(1):94–102, 1995.

[10] D. Lei. Population-based neighborhood search for job shop scheduling with interval processing time. *Computers & Industrial Engineering*, 61(4):1200–1208, 2011.

[11] Y. N. Sotskov, T.-C. Lai, F. Werner. Measures of problem uncertainty for scheduling with interval processing times. *OR Spectrum*, 35(3):659–689, 2013.

[12] A. Allahverdi, H. Aydilek, A. Aydilek. Single machine scheduling problem with interval processing times to minimize mean weighted completion time. *Computers & Operations Research*, 51:200–207, 2014.

[13] A. Allahverdi. A survey of scheduling problems with uncertain interval/bounded processing/setup times. *Journal of Project Management*, 7(4):255–264, 2022.

[14] W. Xu, W. Wu, Y. Wang, Y. He, Z. Lei. Flexible job-shop scheduling method based on interval grey processing time. *Applied Intelligence*, 53:14876–14891, 2023.

[15] H. Díaz, J. J. Palacios, I. Díaz, C. R. Vela, I. González-Rodríguez. Robust schedules for tardiness optimization in job shop with interval uncertainty. *Logic Journal of the IGPL*, 2022. doi:10.1093/jigpal/jzac016.

[16] J. R. Koza. *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press, Cambridge, MA, 1992.

[17] R. Poli, W. B. Langdon, N. F. McPhee. *A Field Guide to Genetic*

Programming. Lulu Enterprises, 2008. Freely available at `http://www.gp-field-guide.org.uk`.

[18] J. Branke, S. Nguyen, C. W. Pickardt, M. Zhang. Automated design of production scheduling heuristics: a review. *IEEE Transactions on Evolutionary Computation*, 20(1):110–124, 2016.

[19] S. Nguyen, Y. Mei, M. Zhang. Genetic programming for production scheduling: a survey with a unified framework. *Complex & Intelligent Systems*, 3(1):41–66, 2017.

[20] D. Karunakaran, Y. Mei, G. Chen, M. Zhang. Dynamic job shop scheduling under uncertainty using genetic programming. In *Intelligent and Evolutionary Systems*, pages 195–210. Springer, 2017.

[21] M. Xu, Y. Mei, F. Zhang, M. Zhang. Learn to optimise for job shop scheduling: a survey with comparison between genetic programming and reinforcement learning. *Artificial Intelligence Review*, 58:160, 2025.

[22] F. J. Gil-Gala, C. Mencía, M. R. Sierra, R. Varela. Learning ensembles of priority rules for online scheduling by hybrid evolutionary algorithms. *Integrated Computer-Aided Engineering*, 28(1):65–80, 2021.

[23] F. J. Gil-Gala, M. Đurašević, M. R. Sierra, R. Varela. Tree search hyper-heuristic with application to combinatorial optimization. *Journal of Heuristics*, 31:25, 2025.

[24] F. J. Gil-Gala, M. Đurašević, M. R. Sierra, J. Puente. Greedy randomised adaptive solution builder using priority rules. In *Proc. Genetic and Evolutionary Computation Conference Companion (GECCO '25)*, pages 211–214. ACM, 2025.

[25] C. Zhang, W. Song, Z. Cao, J. Zhang, P. S. Tan, C. Xu. Learning to dispatch for job shop scheduling via deep reinforcement learning. *NeurIPS*, 2020.

[26] J. Park, J. Chun, S. H. Kim, Y. Kim, J. Park. Learning to schedule job-shop problems: representation and policy learning using graph neural network and reinforcement learning. *International Journal of Production Research*, 59(11):3360–3377, 2021.

[27] J. J. Palacios, I. González-Rodríguez, C. R. Vela, J. Puente. Coevolutionary makespan optimisation through different ranking methods for the fuzzy flexible job shop. *Fuzzy Sets and Systems*, 278:81–97, 2015.

Table A.9: Per-instance RE (%) of the constructive methods (single deterministic pass). LB is the crisp Taillard lower bound.

Inst.	LB	MOR	G&T	GP	Inst.	LB	MOR	G&T	GP
TA1	1231	29.7	26.1	19.3	TA36	1819	57.4	42.8	23.8
TA2	1244	34.8	27.8	14.2	TA37	1771	43.0	35.6	18.4
TA3	1218	42.9	33.2	14.1	TA38	1673	50.6	28.0	20.1
TA4	1175	62.6	27.7	13.0	TA39	1795	50.2	28.9	22.4
TA5	1224	50.7	33.0	14.7	TA40	1651	42.7	42.3	26.1
TA6	1238	36.6	18.2	13.7	TA41	1906	53.5	45.0	33.4
TA7	1227	39.4	26.1	17.7	TA42	1884	61.3	38.1	24.3
TA8	1217	41.3	25.0	14.8	TA43	1809	61.4	39.8	22.3
TA9	1274	38.2	28.2	20.0	TA44	1948	58.8	34.7	25.8
TA10	1241	38.0	22.1	15.4	TA45	1997	59.5	33.7	14.9
TA11	1357	49.9	30.7	17.5	TA46	1957	59.7	31.6	22.4
TA12	1367	47.8	34.6	17.6	TA47	1807	78.3	30.1	25.6
TA13	1342	46.5	26.9	13.3	TA48	1912	52.2	33.7	21.1
TA14	1345	44.1	29.7	13.2	TA49	1931	46.9	35.3	20.1
TA15	1339	51.3	30.6	16.7	TA50	1833	56.2	46.2	31.3
TA16	1360	35.8	34.0	15.6	TA51	2760	38.2	34.7	23.2
TA17	1462	50.1	17.7	13.9	TA52	2756	38.1	26.5	12.7
TA18	1377	54.2	31.8	23.6	TA53	2717	28.2	18.3	11.8
TA19	1332	43.2	29.3	20.0	TA54	2839	23.6	16.1	8.2
TA20	1348	43.7	24.1	15.1	TA55	2679	37.6	29.2	15.5
TA21	1642	40.8	28.0	20.2	TA56	2781	44.8	19.8	10.9
TA22	1561	46.3	30.8	17.4	TA57	2943	29.9	20.0	9.6
TA23	1518	45.2	35.3	17.8	TA58	2885	30.5	26.3	13.6
TA24	1644	36.2	23.8	14.1	TA59	2655	45.2	28.2	16.7
TA25	1558	53.2	33.8	16.6	TA60	2723	25.3	19.8	15.5
TA26	1591	52.4	31.2	26.9	TA61	2868	48.7	25.4	15.4
TA27	1652	49.8	31.1	21.4	TA62	2869	43.5	27.6	17.2
TA28	1603	38.8	26.8	12.4	TA63	2755	46.0	23.8	13.4
TA29	1583	56.4	32.0	17.2	TA64	2702	43.3	28.4	12.2
TA30	1528	53.3	36.1	17.0	TA65	2725	44.7	30.0	20.5
TA31	1764	34.4	34.4	24.5	TA66	2845	38.1	28.0	11.9
TA32	1774	55.8	33.3	19.0	TA67	2825	58.4	26.1	15.2
TA33	1788	41.6	34.8	27.5	TA68	2784	45.8	19.1	10.6
TA34	1828	34.8	31.8	20.7	TA69	3071	38.0	22.8	12.9
TA35	2007	30.5	24.1	8.4	TA70	2995	52.3	23.4	16.7